Old Dominion University

ODU Digital Commons

Cybersecurity Undergraduate Research Showcase

2026 Spring Cybersecurity Undergraduate Research Projects

Hijacking the Prompt: A Survey of Prompt Injection Attacks, Detection, and Defense in Large Language Models

Edward J. Griggs
Christopher Newport University

Follow this and additional works at: https://digitalcommons.odu.edu/covacci-undergraduateresearch

 Part of the Artificial Intelligence and Robotics Commons, Cybersecurity Commons, and the Information Security Commons

Griggs, Edward J., "Hijacking the Prompt: A Survey of Prompt Injection Attacks, Detection, and Defense in Large Language Models" (2026). *Cybersecurity Undergraduate Research Showcase*. 9.
https://digitalcommons.odu.edu/covacci-undergraduateresearch/2026spring/projects/9

This Paper is brought to you for free and open access by the Undergraduate Student Events at ODU Digital Commons. It has been accepted for inclusion in Cybersecurity Undergraduate Research Showcase by an authorized administrator of ODU Digital Commons. For more information, please contact digitalcommons@odu.edu.

Prompt Injection Attacks in Large Language Models: A Survey of Attack Taxonomies, Risks, Detection, and Prevention

Edward Griggs

B.S. in Cybersecurity

Christopher Newport University

Mentor: Yan Lu

Christopher Newport University

April 17, 2026

Abstract

This survey examines prompt injection attacks, one of the more pressing security issues affecting large language models (LLMs) and the applications built around them. As LLMs have become embedded in enterprise workflows, agentic systems, and consumer tools, prompt injection has emerged as a threat that puts the integrity, confidentiality, and reliability of AI systems at real risk. This paper builds out a taxonomy of attack vectors spanning direct injection, indirect injection, multimodal attacks, tool and agent exploitation, hybrid chained techniques, and autonomous propagating threats. It also examines the risk landscape across five impact categories: data exfiltration, unauthorized actions, content manipulation, system compromise, and supply chain propagation. Detection approaches covered include heuristic methods, machine learning classification, perplexity analysis, canary tokens, and LLM-as-judge architectures. Prevention strategies are analyzed at the model, application, and system levels. Drawing on 18 primary sources from 2023 to 2025, the main takeaway is that prompt injection remains largely unsolved. Current LLM architectures cannot reliably distinguish instructions from data, and that fundamental gap drives the entire threat landscape. The survey closes with recommendations for layered defense strategies and flags the areas where future research is most urgently needed.

Introduction

Large language models have changed the landscape of artificial intelligence in a pretty short amount of time. We've moved from basic text generation to models that can write code, plan multi-step tasks, and take real-world actions through integrated tools. That's genuinely impressive, but it also means the security stakes have gotten much higher. When an LLM is integrated into healthcare systems, financial platforms, or government services, a vulnerability in how it processes inputs isn't just a technical quirk. It can have serious, real-world consequences.

Prompt injections are one of those vulnerabilities. First identified as its own class of attack in 2022, it exploits something fundamental about how LLMs work: they cannot reliably separate trusted instructions from untrusted user data when both show up in the same input stream. It's a lot like SQL injection in web security, where failing to separate code from data lets an attacker sneak command into what should just be content. In the LLM context, any instruction-like text in the context window can potentially be acted on, regardless of where it came from.

The OWASP Foundation ranked prompt injection as the number one vulnerability in their 2025 Top 10 for Large Language Model Applications, which reflects how seriously the security community takes this threat (OWASP Foundation, 2025). NIST also included it in their 2025 taxonomy of adversarial machine learning attacks (NIST, 2025). The problem has grown in scope as agentic AI systems have become more widespread, meaning models that don't just generate text but call APIs, browse the web, and execute code. In those environments, a successful prompt injection doesn't just manipulate output. It can trigger unauthorized actions, leak sensitive data, or compromise connected systems.

This survey covers prompt injection across four areas: attack taxonomy, risk analysis, detection techniques, and prevention strategies. It draws on 18 primary sources, including academic papers, industry reports, and government standards published between 2023 and 2025, with the goal of giving a clear picture of where the threat stands and what current defenses can realistically offer.

Literature Review

The following review is organized around four areas that reflect the current state of knowledge on prompt injection: attack taxonomy and classification, risk analysis and impact assessment, detection techniques, and prevention strategies. Each section draws from multiple sources to build a balanced view.

Attack Taxonomy and Classification

The classification of prompt injection attacks has evolved quickly as the attack surface has expanded alongside LLM capabilities. The taxonomy used in this survey covers six hierarchical categories synthesized from the comprehensive review by Abdallah et al. (2025), the hybrid threat framework from Ahmed et al. (2025), and the foundational indirect injection work by Greshake et al. (2023).

Direct injection covers attacks where a user explicitly crafts malicious input through the primary prompt channel. These are the original and most well-studied class of prompt injection, and they all come down to the same problem: the model cannot distinguish instructions from data when both arrive through the same channel. The main techniques include jailbreaking (convincing the model to adopt an unrestricted persona, like the well-known 'DAN' prompts), goal hijacking

(redirecting the model's objective via phrases like 'ignore previous instructions and instead...'), and prompt leaking (extracting system prompts to understand and target the application's guardrails for follow-up attacks).

Indirect injections are where things get more dangerous in practice. Instead of crafting malicious input directly, the attacker plants instructions in external content that the LLM will eventually retrieve and process. This class was formalized by Greshake et al. (2023) and represents arguably the most serious threat vector for real-world applications, because the attacker never needs to touch the model interface. Key techniques include RAG poisoning (injecting instructions into knowledge base documents that get retrieved at query time), web content injection (embedding payloads in web pages that LLM-powered agents will process), and email and document injection (placing instructions in files that AI assistants summarize or act on).

Multimodal injections emerged as a distinct category in 2024-2025 as vision and audio-capable LLMs became widely deployed. These attacks embed instructions in images or audio in ways humans cannot easily perceive but models interpret as actionable commands, including text at near-invisible opacity, adversarial image perturbations, and ultrasonic audio commands targeting voice assistants.

Tool and agent-based injection is the frontier category as of 2024-2025. Agentic AI systems with access to external tools, APIs, and file systems expand the attack surface beyond text manipulation into real-world action. MCP tool poisoning compromises Model Context Protocol server responses to redirect agent behavior, while function call manipulation tricks agents into invoking tools with attacker-controlled parameters and potentially calling destructive operations the user never authorized.

Hybrid and chained injection techniques combine multiple vectors to bypass layered defenses. Ahmed et al. (2025) formalize these as 'Prompt Injection 2.0,' meaning attacks that distribute malicious instructions across multiple turns (where no single turn triggers detection) or combine direct and indirect channels to exploit trust relationships. Finally, autonomous and propagating injections represent the most severe theoretical class: self-replicating payloads, demonstrated by the Morris II 'AI worm' concept, that spread through LLM-integrated systems without continued attacker involvement by instructing the compromised model to reproduce the payload in its outputs.

Risk Analysis and Impact Assessment

The risks from prompt injections go well beyond getting the model to generate inappropriate content. As LLMs have been integrated with tools, APIs, and real decision-making workflows, successful injection attacks can cause harm across five distinct dimensions.

Data exfiltration is one of the highest-severity outcomes. Prompt injections can cause LLMs to expose system prompts, user data, and proprietary business logic, often without the user's awareness. Greshake et al. (2023) demonstrated a particularly vivid example: a poisoned document instructs an AI assistant to encode confidential data into a URL rendered as a markdown image, triggering an HTTP request that silently transmits the data to an attacker-controlled server. Willison (2024) has documented real-world system prompt extractions from ChatGPT custom GPTs using nothing more than 'repeat your instructions verbatim.'

Unauthorized actions become possible when LLMs have tool access. A successful injection can cause the model to send emails, modify files, make purchases, or call APIs with attacker-controlled parameters (Abdallah et al., 2025). The severity scales directly with the agent's

permission scope. A model with file system and network access is a fundamentally different risk profile than a text-only chatbot. In agentic and tool-augmented deployments, this can escalate all the way to system compromise: arbitrary code execution, privilege escalation, or persistent backdoor installation (CrowdStrike, 2024).

Content manipulation is more subtle but also significant. Attackers can bias or corrupt model output in ways the user never detects, since users tend to treat AI responses as objective and authoritative (Yi et al., 2024). Greshake et al. (2023) showed how indirect injection via web pages could bias Bing Chat responses about specific products or people. Supply chain propagation is the hardest risk to reason about but potentially the most consequential. When a compromised model's output becomes input to another system, through shared knowledge bases, AI-generated code, or multi-agent workflows, a single injection can cascade beyond any single organization's control (Ahmed et al., 2025).

Detection Techniques

Detecting prompt injections is genuinely difficult. There's no clean rule that separates adversarial instructions from legitimate user input in natural language, and current detection approaches all involve real tradeoffs between accuracy, speed, and coverage.

Heuristic and rule-based detection scans inputs for known injection signatures like 'ignore previous instructions' using keyword lists and regex patterns. It's fast and interpretable, but trivially bypassed through paraphrasing, encoding tricks like Base64 or ROT13, or just switching languages. The Rebuff heuristic layer exemplifies this approach (ProtectAI, 2023), but its effectiveness is essentially limited to attacks that match its existing ruleset, and that ruleset requires constant maintenance as new patterns emerge (Schulhoff et al., 2023).

ML classification uses trained models to distinguish benign from adversarial inputs. Lakera Guard and Rebuff's vector similarity layer both use this approach, achieving higher detection rates than heuristics for known patterns (Liu et al., 2024). The limitation is training data dependency: novel attacks outside the distribution consistently slip through. Perplexity analysis takes a statistical approach, flagging inputs with anomalous token distributions. It's useful against machine-generated adversarial suffixes, as Alon and Kamfonas (2023) demonstrated, but largely ineffective against human-crafted prompts that have natural perplexity scores.

Canary token detection plants secret strings in the system prompt and monitors outputs for their appearance. It's a proactive approach that detects successful context leakage but does nothing against goal hijacking or tool manipulation (ProtectAI, 2023; Abdallah et al., 2025). LLM-as-judge detection uses a secondary model to evaluate whether inputs attempt to subvert intended behavior, offering the most capable detection for semantically sophisticated attacks. The downsides are real though: it adds latency and cost, and the judge model is itself vulnerable to injection, which becomes more severe when the judge is weaker than the primary model (Yi et al., 2024).

Prevention Strategies

Prevention operates at three levels: model-level defenses, application-level controls, and system-level architectural patterns. Each addresses a different part of the attack surface, and none is sufficient on its own.

Instruction hierarchy is the most theoretically sound model-level defense. Formalized by Wallace et al. (2024) at OpenAI, it trains LLMs to assign privilege levels to instructions based on their source, with system prompts at the top, user messages in the middle, and retrieved content at the bottom. Anthropic implements a similar approach in Claude. This is highly effective against

direct injection but partially bypassable through indirect injection, where malicious content arrives through trusted data channels. More importantly, it requires model-level training changes, which means only model providers can actually implement it.

Application-level defenses include input sanitization (stripping known injection patterns, normalizing Unicode, and enforcing length limits) and output validation (scanning outputs for sensitive data leakage, format violations, and canary token appearances). These mirror traditional web security practices like XSS filtering and SQL parameterization, useful layers but unable to provide complete protection given the inherent ambiguity between instructions and data in natural language. Guardrail frameworks like NVIDIA NeMo Guardrails and Lakera Guard provide configurable middleware that combines multiple detection and filtering approaches in a single deployable layer.

System-level defenses focus on limiting damage rather than preventing injection. Sandboxing and least-privilege constraints on tool access mean that even a successful injection can only do what the agent was already permitted to do. Dual-LLM architectural patterns, where a quarantined model handles untrusted input and a privileged model makes decisions, provide the strongest theoretical protection by removing the LLM from the critical execution path. The tradeoff is significantly complex and reduced autonomy. Human-in-the-loop confirmation for irreversible or high-impact operations provides a more pragmatic middle ground.

Methodology

This research uses a structured literature survey methodology to examine the current state of prompt injection attacks and defenses. Source selection covered four types of publications: peer-reviewed academic papers, industry reports from cybersecurity firms and AI companies,

government standards and frameworks, and documentation from open-source security tools. A total of 18 primary sources published between 2023 and 2025 were selected based on relevance, credibility, and recency. Academic papers were weighted more heavily for theoretical foundations and empirical findings, while industry and government sources provided practical context and standardized frameworks.

The analysis is organized around four pillars, covering attack taxonomy, risk analysis, detection, and prevention, with each drawing from multiple sources to allow cross-validation. The taxonomy started with established categories from Greshake et al. (2023) and was extended to incorporate newer attack vectors documented in 2024-2025 sources.

A few limitations are worth being upfront about. Publication bias favors well-documented attack vectors, so novel or undisclosed techniques won't appear in the literature. The field moves fast enough that some findings may already be outdated. Effectiveness claims for many defensive tools are hard to verify independently given the lack of standardized benchmarks. And this survey is scoped specifically to prompt injections; it doesn't cover adjacent concerns like training data poisoning or model extraction attacks.

Discussion

One of the clearest patterns in the research is that prompt injection has evolved in direct lockstep with LLM capabilities. Every new capability added to these systems, whether that's tool use, web browsing, or multi-agent orchestration, has created a corresponding new attack surface, and adversaries have adapted quickly (Ahmed et al., 2025). That trajectory strongly suggests future LLM capabilities will keep introducing novel injection vectors, which means static defense

approaches are going to keep falling behind. The only viable response is building adaptive, layered defenses that can evolve alongside the capabilities they're protecting.

The relationship between detection gaps and risk severity is worth focusing on. Current detection architectures face a structural disadvantage: novel injection techniques can be developed faster than detection systems can be updated (Abdallah et al., 2025). This directly enables the most severe outcomes, including data exfiltration through markdown image encoding and unauthorized tool invocation through indirect injection. It's honestly similar to the early antivirus industry, where signature-based detection kept falling behind new threats. The prompt injection field appears to be in that same reactive phase right now.

Prevention analysis reveals a real tension between security and utility, and highlights how responsibility is fragmented across different stakeholders. The instruction hierarchy approach (Wallace et al., 2024) is the most theoretically sound defense but is only available to model providers. Application developers are left with input sanitization, guardrail frameworks, and output validation. System architects can add sandboxing and least-privilege constraints on top of that. The emerging consensus is a three-tier division: model providers handle instruction hierarchy, developers deploy guardrails, and architects enforce isolation. Supply chain risks make this harder, as the Morris II AI worm concept showed that a single injection can cascade beyond any single organization's ability to contain it (Ahmed et al., 2025). The practical takeaway is that organizations deploying LLM-integrated systems need to treat prompt injection testing with the same rigor as traditional web application penetration testing and build incident response plans that account for AI-specific attack characteristics.

Conclusion

This survey examined prompt injections across four dimensions, covering taxonomy, risks, detection, and prevention, and the overall conclusion is fairly clear: prompt injection is a fundamental, largely unsolved security challenge. Until LLMs can reliably distinguish between instructions and data in their context window, the vulnerability isn't going away.

The attack taxonomy has grown significantly from its 2022 origins. What started as direct injection techniques now encompasses indirect injection through external data sources, multimodal attacks via vision and audio, tool and agent exploitation, hybrid chained attacks, and autonomous propagating threats. Each expansion of LLM capability has brought a corresponding expansion of the attack surface.

Current defenses, including instruction hierarchy, input and output filtering, guardrail frameworks, and sandboxing, provide meaningful risk reduction when properly combined. But the fundamental vulnerability persists as an inherent property of how transformer models treat all context window content as equally authoritative. Defense-in-depth is the right posture, not a search for a single fix.

Future research needs to prioritize formal verification methods for LLM instruction compliance, building on Liu et al. (2024). The dual-LLM architectural pattern also deserves more serious evaluation as a production-viable defense. Extending instruction hierarchy training to handle indirect injections through trusted data channels remains one of the most important open problems in the field. Until those advances arrive, the right approach is to treat prompt injection as a persistent, active threat that requires continuous monitoring, testing, and adaptive defense strategies.

References

Abdallah, M., Barreno, A., Bensalah, F., et al. (2025). Prompt injection attacks in large language models and AI agent systems: A comprehensive review. Information, 17(1), 54. https://www.mdpi.com/2078-2489/17/1/54

Ahmed, S., Kaur, K., & Sharma, R. (2025). Prompt injection 2.0: Hybrid AI threats. arXiv. https://arxiv.org/abs/2507.13169

Alon, G., & Kamfonas, M. (2023). Detecting language model attacks with perplexity. arXiv. https://arxiv.org/abs/2308.14132

Anthropic. (2024a). Research on AI safety and prompt security. Anthropic. https://www.anthropic.com/research

Anthropic. (2024b). System prompt security best practices. Anthropic. https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/system-prompts

BSG. (2025). OWASP LLM Top 10 (2025) analysis. BSG Tech Blog. https://bsg.tech/blog/owasp-llm-top-10/

CrowdStrike. (2024). Prompt injection: Definition and attack taxonomy. CrowdStrike Cybersecurity 101. https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/prompt-injection/

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Giner, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv. https://arxiv.org/abs/2302.12173

Lakera AI. (2024). Lakera Guard: Real-time prompt injection detection. Lakera AI. https://www.lakera.ai/

Liu, Y., Jia, Y., Geng, R., Jia, J., & Gong, N. Z. (2024). Formalizing and benchmarking prompt injection attacks and defenses. arXiv. https://arxiv.org/abs/2310.12815

National Institute of Standards and Technology. (2025). Adversarial machine learning: A taxonomy and terminology of attacks and mitigations (AI 100-2e2025). NIST. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf

NVIDIA. (2024). NeMo Guardrails: Programmable guardrails for LLM applications. GitHub. https://github.com/NVIDIA/NeMo-Guardrails

OWASP Foundation. (2025). OWASP Top 10 for large language model applications. OWASP. https://owasp.org/www-project-top-10-for-large-language-model-applications/

ProtectAI. (2023). Rebuff: Self-hardening prompt injection detector. GitHub. https://github.com/protectai/rebuff

Schulhoff, S., Pinto, J., Khan, A., Bouchard, L.-F., Si, C., Anati, S., & Boyd-Graber, J. (2023). Ignore this title and HackAPrompt: Exposing systemic weaknesses of LLMs through a global scale prompt hacking competition. arXiv. https://arxiv.org/abs/2311.16119

Wallace, E., Xiao, K., Leike, J., Weng, L., Heidecke, J., & Beutel, A. (2024). The instruction hierarchy: Training LLMs to prioritize privileged instructions. arXiv. https://arxiv.org/abs/2404.13208

Willison, S. (2024). Prompt injection series. Simon Willison's Weblog. https://simonwillison.net/series/prompt-injection/

Yi, J., Xie, Y., Zhu, B., Hines, K., Kiciman, E., Sun, G., Xie, X., & Wu, F. (2024). Jailbreak attacks and defenses against large language models: A survey. arXiv. https://arxiv.org/abs/2407.04295